How To: Maintain the Project

Use this guide when you change the engine, the app, or the docs. Each code change gets its documentation change in the same pass.

How to run the Python test suite

```
cd tests
python -m pytest -v
```

All tests must pass before you ship a change. If a test fails, fix the code or the test. Do not delete a failing test to make the suite pass. `README.md` states the current test count.

How to check the JavaScript app matches the Python engine

The app (`app/index.html`) is a hand-written copy of the Python engine. It is not generated from the Python code. The parity script checks the copy:

```
node tests/js_parity.js
```

The script extracts the engine section of the `<script>` block in `app/index.html`. It runs the exact Tier 1, Tier 2 and behavior functions against the hand-derived values of the Python tests. It checks the Monte Carlo Daily Double functions for invariants only, because the two engines use different random-number generators. CI runs this script on every push and pull request (`.github/workflows/ci.yml`).

If you change a value in the configuration block of `engine/behavior.py`, change the same value in the `BEHAVIOR` object in `app/index.html`. Then run the parity script.

How to regenerate the PDF companion for a doc

Each Markdown doc in this project has a PDF companion in `build/`. If you edit a `.md` file, regenerate its PDF in the same pass.

```
python build/md_to_pdf.py &lt;source.md&gt; build/&lt;name&gt;.pdf
```

Example:

```
python build/md_to_pdf.py docs/reference/engine-api.md build/reference-engine-api.pdf
```

The script uses Python-Markdown (`pip install markdown`). It renders the PDF with WeasyPrint. If WeasyPrint cannot load its GTK libraries (common on Windows), the script uses headless Microsoft Edge or Google Chrome. The script prints the renderer that it used. Pandoc is not necessary.

To check a PDF, extract its first page:

```
pdf-inspector build/<name>.pdf --pages 1
```

How to update the changelog and risk register

1. Add a new entry at the top of `CHANGELOG.md`, in Keep a Changelog format, with an ISO date (`YYYY-MM-DD`). Bump the version. Include a `### Record` section with the fields **Measured**, **Basis**, **Supersedes** and **Owed to the next major**.
2. If the change resolves, changes, or adds a risk, update `RISK_REGISTER.md` in the same pass. Do not delete an old risk's row to mark it resolved — update its status column and explain why.
3. If the change reverses an earlier documented conclusion (in the white paper, an ADR, or a past changelog entry), state what changed and why in the changelog entry. If a published figure or statement is no longer true, delete it from each document in the same pass. Do not add a caption to it. An ADR that a new ADR replaces keeps its body as the historical record; change only its Status line.
4. Check that the version number at the top of `CHANGELOG.md` matches the version stamped in `docs/white-paper.md`.

How to check everything before you call a change "done"

Do this checklist, in sequence:

1. `python -m pytest -v` from `tests/` — all tests pass.
2. `node tests/js_parity.js` — all checks pass, if you changed the engine or the app.
3. `CHANGELOG.md`, `RISK_REGISTER.md`, and `docs/white-paper.md` are updated, if the change affects behavior or a documented conclusion.
4. Each changed `.md` file has a regenerated PDF in `build/`.
5. The cross-project artifact audit (`python ../portfolio/build/check_projects.py`) passes. It fails if a required artifact is missing or if the newest changelog version does not match the white paper version.
6. Local files, GitHub (`origin`, https://github.com/willhoop/jeopardy-wagering) and the live app (https://willhoop.github.io/jeopardy-wagering/app/) agree.